

Applications of Binary Tree: Expression Tree & Huffman Encoding -- Data Structures

Definition: Binary Trees are applied extensively in compilers (Expression Trees for arithmetic parsing) and data compression algorithms (Huffman Encoding for prefix-free lossless entropy coding).

1. Application 1: Expression Trees

An Expression Tree is a binary tree where internal nodes are operators (+, -, *, /) and leaf nodes are operands (a, b, 5, 10).

Expression: (a + b) * (c - d)



Traversals of Expression Tree:

1. Inorder Traversal: Yields Infix Expression with parentheses -> ((a + b) * (c - d))

2. Pre

Constructing Expression Tree from Postfix Expression:

1. Read postfix string token by token.

2. If t

2. Application 2: Huffman Encoding (Data Compression)

Huffman Coding is a greedy algorithm that assigns variable-length binary codes to characters based on their frequency of occurrence. More frequent characters get shorter codes.

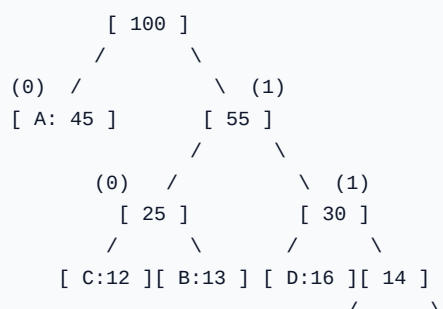
Step-by-Step Huffman Tree Construction:

1. Count frequency of each unique character in input text.

2. Ins

Sample Character Frequencies:

A: 45, B: 13, C: 12, D: 16, E: 9, F: 5



[F:5] [E:9]

Resulting Prefix-Free Codes:

* A -> 0 (1 bit)

* C -

3. Must-Write Points for Exams

* Huffman codes are Prefix-Free Codes (no character code is a prefix of any other code), enabling unambiguous decompression.

* In

4. Quick Recall Flow

Expression Tree: Leaves=Operands, Internal=Operators | Huffman Coding: Min-Heap -> Merge 2 Lowest Frequencies -> 0
Left / 1 Right -> Variable Length Prefix Code